

Built with Claude: `par ( ... )` — make a for-loop run in parallel

loft is a programming language I've been building for 9 years — recently with Claude as a collaborator. Over the past two months we worked on several things; the parallel for-loop below was one of the bigger pieces. Here's how it compares to Java and Go.

loft

```
fn process(d: const Document) -> Report { ... }
fn handle(r: Report) { ... }

fn main() {
  for x in work par(r = process(x), 4) {
    handle(r);
  }
}
```

Java

```
static Report process(Document d) { ... }
static void handle(Report r) { ... }

work.stream()
  .parallel()
  .map(Main::process)
  .forEach(Main::handle);
```

Go

```
func process(d Document) Report { ... }
func handle(r Report) { ... }

results := make(chan Report, len(work))
var wg sync.WaitGroup
chunk := len(work) / workers
for t := 0; t < workers; t++ {
  wg.Add(1)
  go func(t int) {
    defer wg.Done()
    lo, hi := t*chunk, (t+1)*chunk
    if t == workers-1 { hi = len(work) }
    for _, x := range work[lo:hi] {
      results <- process(x)
    }
  }(t)
}
go func() { wg.Wait(); close(results) }()

for r := range results {
  handle(r)
}
```

Language	What you write	What is hard or impossible
loft	one extra clause on the for-loop	nothing — the safety rules are checked by the compiler
Java	one method call	controlling thread count or how work is split
Go	a hand-built scaffold	writing it concisely — the type system can't catch a worker that touches the wrong data

Four lessons from a long Claude collaboration

Two months of pair-programming. The lessons were not about parallel code.

2

months

3

plan rewrites

14+

hidden bugs found

-565

FUN FACT

lines, biggest single delete

1 The wrong version teaches you the right one

We rewrote the planning document three times in eight days because each version eventually drifted from reality. The same is true of code: it is fine to let Claude implement something you suspect will need to be redone. **Let Claude write the imperfect version first; the tests and the bug list survive the rewrite.**

3 Sometimes the side-effect is the point

The plan was to delete a thousand lines of complicated code. But the small test programs we wrote along the way kept finding completely different bugs in places no normal test would touch. **File every bug Claude's work surfaces, even the unrelated ones — they're the cheapest you'll ever find.**

2 Evidence is cheaper than design

Claude's instinct is to design from the problem description. For one bug that produced a ~200-line plan with new compiler concepts. Thirty seconds reading the actual error message collapsed it to a 30-line routing change. **Make Claude paste the actual error before it writes the fix.**

4 A 9-year project does not optimise for “ship this week”

I had to say no to marking failing tests as “allowed to fail”, to branching off main while work was unfinished, and to accepting small slowdowns for short-term wins. **The second time you tell Claude “no” to the same thing, save it to memory.**

The interesting part of this collaboration wasn't the parallel code. It was discovering what structure a long-running Claude session needs around it.